

HUMAN ACTIVITY AND POSTURAL TRANSITION RECOGNITION

Deep Learning-Based Comparative Evaluation Using Smartphone Inertial
Sensors

FINAL PROJECT REPORT

Student Name: Ahmed Hisham

Student ID: 223106291

Course / Department: Deep Learning AIE 332

Supervisor: Dr. Shaker Elsappagh

University: Galala University

2026

Table of Contents

Abstract

1. Introduction

2. Dataset and Data Preparation

2.1 Dataset Source

2.2 Input Representation

2.3 Subject-Based Split

2.4 Windowing

2.5 Class Distribution

3. Preprocessing and Data Loading

3.1 Normalization

3.2 PyTorch DataLoaders

3.3 Computational Environment

4. Deep Learning Models

4.1 CNN

4.2 LSTM

4.3 GRU

4.4 CNN-LSTM

4.5 Transformer

5. Training Configuration

6. Training and Validation Analysis

7. Test Results and Model Comparison

7.1 Overall Performance

8. CNN Classification Analysis

8.1 Confusion Matrix

8.2 CNN Classification Report

9. Per-Class Comparison Across Models

10. Sensor Ablation Study

11. Efficiency and Ranking

11.1 Final Rankings

12. Discussion

12.1 Best Overall Model

12.2 CNN-LSTM Trade-off

12.3 Recurrent Models

12.4 Transformer

12.5 Class Imbalance

13. Limitations and Future Work

14. Conclusion

15. References

Appendix A. Reproducibility Summary

List of Figures

Figure 1. Training and Validation Accuracy

Figure 2. Training and Validation Loss

Figure 3. CNN Confusion Matrix

Figure 4. Per-Class F1-Score Comparison

Figure 5. Test Accuracy vs Model Complexity

Figure 6. Test Accuracy vs Training Time

Figure 7. Sensor Ablation Performance

List of Tables

Table 1. Project Dataset Split

Table 2. Activity Classes and Window Counts

Table 3. Model Architectures and Trainable Parameters

Table 4. Common Training Configuration

Table 5. Final Model Comparison

Table 6. CNN Per-Class Classification Results

Table 7. Sensor Ablation Results

Table 8. Final Model Rankings

Abstract

This project investigates deep-learning approaches for smartphone-based Human Activity Recognition (HAR) using raw accelerometer and gyroscope signals. The project uses the Smartphone-Based Recognition of Human Activities and Postural Transitions dataset and constructs 128-sample temporal windows from six sensor channels: accelerometer X/Y/Z and gyroscope X/Y/Z. The resulting processed dataset contains 5,773 windows across 12 activity and postural-transition classes. A subject-based split assigns users 1–21 to training, 22–25 to validation, and 26–30 to testing.

Five neural architectures were implemented in PyTorch: a one-dimensional CNN, LSTM, GRU, CNN-LSTM, and Transformer encoder. All models were trained for 15 epochs using CrossEntropyLoss and Adam with a learning rate of 0.001. Performance was evaluated using accuracy, precision, recall, weighted F1, Macro F1, training time, inference time, and trainable parameter count.

The CNN using both sensors achieved the strongest overall performance, with 93.73% test accuracy, 93.99% precision, 93.73% recall, 93.65% weighted F1, and 86.76% Macro F1 using 44,620 trainable parameters. The sensor-ablation experiment further showed that accelerometer-only input achieved 91.90% accuracy, while gyroscope-only input achieved 72.23%; combining both sensors restored the best overall performance. These results indicate that the CNN provides the best balance of accuracy, class-balanced performance, and computational efficiency among the tested configurations.

Keywords: Human Activity Recognition, Smartphone Sensors, Accelerometer, Gyroscope, CNN, LSTM, GRU, Transformer, Deep Learning.

1. Introduction

Human Activity Recognition is the automatic classification of human movements from sensor measurements. Smartphones provide inertial sensors that can capture movement without requiring specialized external hardware. The project focuses on raw inertial signals rather than relying on a precomputed feature vector, allowing the deep-learning models to learn temporal representations directly from the sensor sequences.

The selected dataset contains six basic activities—walking, walking upstairs, walking downstairs, sitting, standing, and laying—and six postural transitions: stand-to-sit, sit-to-stand, sit-to-lie, lie-to-sit, lie-to-stand, and stand-to-lie. The official UCI repository describes the raw accelerometer and gyroscope signals as three-axis measurements collected at 50 Hz from smartphones worn at the waist. [1]

The central experimental question is which deep-learning architecture provides the strongest recognition performance while keeping model complexity and computation practical. A second question is how much each sensor modality contributes to recognition performance.

2. Dataset and Data Preparation

2.1 Dataset Source

The project uses the UCI Smartphone-Based Recognition of Human Activities and Postural Transitions dataset. The repository provides raw accelerometer files, raw gyroscope files, and a labels file containing experiment, subject, activity, start, and end indices. [1]

The project notebook loads the RawData directory and reads files named in the form acc_expXX_userYY.txt and gyro_expXX_userYY.txt, then combines the two three-axis signals into six sensor features. The notebook reports 123 files in the RawData directory and confirms the combined sensor representation has six channels.

2.2 Input Representation

Each sample is represented as a sequence of 128 time steps. The six channels are ordered as accelerometer X, Y, Z followed by gyroscope X, Y, Z. The complete processed dataset has shape (5773, 128, 6).

Split	Samples	Input Shape
Training	3,910	(3910, 128, 6)
Validation	826	(826, 128, 6)
Test	1,037	(1037, 128, 6)
Total	5,773	(5773, 128, 6)

Table 1. Project dataset split and input dimensions.

2.3 Subject-Based Split

The notebook explicitly separates subjects rather than randomly mixing individual windows: users 1–21 are used for training, users 22–25 for validation, and users 26–30 for testing. This produces the reported 3,910, 826, and 1,037 samples respectively.

2.4 Windowing

For each labeled activity segment, the notebook creates fixed-length sequences of 128 samples using a stride of 128. Activity IDs are converted from the original 1–12 range to zero-based class IDs 0–11. The resulting windows are therefore non-overlapping within each labeled segment.

2.5 Class Distribution

Class ID	Activity	Windows
0	WALKING	893
1	WALKING_UPSTAIRS	809
2	WALKING_DOWNSTAIRS	747
3	SITTING	928
4	STANDING	1019
5	LAYING	1013
6	STAND_TO_SIT	56
7	SIT_TO_STAND	33
8	SIT_TO_LIE	68

9	LIE_TO_SIT	62
10	LIE_TO_STAND	84
11	STAND_TO_LIE	61

Table 2. Activity classes and number of processed windows.

The distribution is highly imbalanced. The six common activities contain hundreds of windows each, whereas the postural-transition classes contain substantially fewer samples. This imbalance is important when interpreting overall accuracy and weighted F1, and it motivates the use of Macro F1 as an additional metric.

3. Preprocessing and Data Loading

3.1 Normalization

Feature normalization is performed using statistics calculated only from the training data. The notebook computes the mean and standard deviation over the training set for each of the six channels, then applies the same values to the training, validation, and test sets. After normalization, the training data have approximately zero mean and unit standard deviation for every channel.

Channel	Training Mean	Training Std.
Accelerometer X	0.81024464	0.40087400
Accelerometer Y	0.01819943	0.42017309
Accelerometer Z	0.06453241	0.33842082
Gyroscope X	0.00611857	0.43447253
Gyroscope Y	-0.00490566	0.41123155
Gyroscope Z	-0.00533715	0.29898904

Table 3. Training-set statistics used for normalization.

3.2 PyTorch DataLoaders

The arrays are converted to float32 tensors for inputs and long tensors for class labels. A batch size of 64 is used. The training DataLoader is shuffled, while validation and test DataLoaders preserve their order.

3.3 Computational Environment

The notebook reports that the models were trained on CPU for the recorded experiments. Training and inference times in this report therefore describe the specific execution environment used for the project and should not be interpreted as hardware-independent benchmarks.

4. Deep Learning Models

Five architectures were implemented with a common 12-class output layer. The implementations differ in how they extract temporal representations from the 128-step sensor sequences.

4.1 CNN

The CNN transposes the input from [batch, time, features] to [batch, features, time], then applies Conv1d(6,64,5), ReLU, MaxPool1d(2), Conv1d(64,128,5), ReLU, and MaxPool1d(2). Adaptive average pooling reduces the temporal dimension to one value per channel before the final Linear(128,12) classifier.

4.2 LSTM

The LSTM receives the six-channel sequence directly and uses a two-layer LSTM with hidden size 64. The output at the last time step is passed to a Linear(64,12) classification layer.

4.3 GRU

The GRU follows the same input/output strategy as the LSTM but replaces the recurrent unit with a two-layer GRU with hidden size 64. The final time-step output is classified by Linear(64,12).

4.4 CNN-LSTM

The CNN-LSTM first applies Conv1d(6,64,5), ReLU, MaxPool1d(2), and Conv1d(64,64,5) with ReLU. The resulting sequence is transposed back to [batch, time, features] and passed to a two-layer LSTM with hidden size 64. The final LSTM time step is classified by Linear(64,12).

4.5 Transformer

The Transformer first projects the six sensor features to a 64-dimensional representation. A TransformerEncoder with two encoder layers and four attention heads processes the sequence. Each encoder layer uses the PyTorch TransformerEncoderLayer configuration, including a 2048-dimensional feed-forward subnetwork and dropout of 0.1. The model then averages the encoder output across time and applies Linear(64,12).

Model	Main architecture	Trainable parameters
CNN	2 Conv1d blocks + pooling + adaptive average pooling	44,620
LSTM	2-layer LSTM, hidden size 64	52,492
GRU	2-layer GRU, hidden size 64	39,564
CNN-LSTM	2 Conv1d stages + 2-layer LSTM	89,868
Transformer	Projection + 2-layer Transformer encoder, 4 heads	563,532

Table 4. Model architectures and trainable parameters.

5. Training Configuration

Setting	Configuration
Input sequence	128 time steps $\times$ 6 features
Output classes	12
Batch size	64
Epochs	15
Loss function	CrossEntropyLoss
Optimizer	Adam

Learning rate	0.001
Weight decay	0
Device reported by notebook	CPU

Table 5. Common training configuration.

The same general training procedure is used for the five main models. The training function records training loss, validation loss, training accuracy, and validation accuracy for every epoch. Training time is measured around the 15-epoch training call.

6. Training and Validation Analysis

The notebook records learning curves for all five models. These figures are reproduced directly from the notebook's embedded outputs.

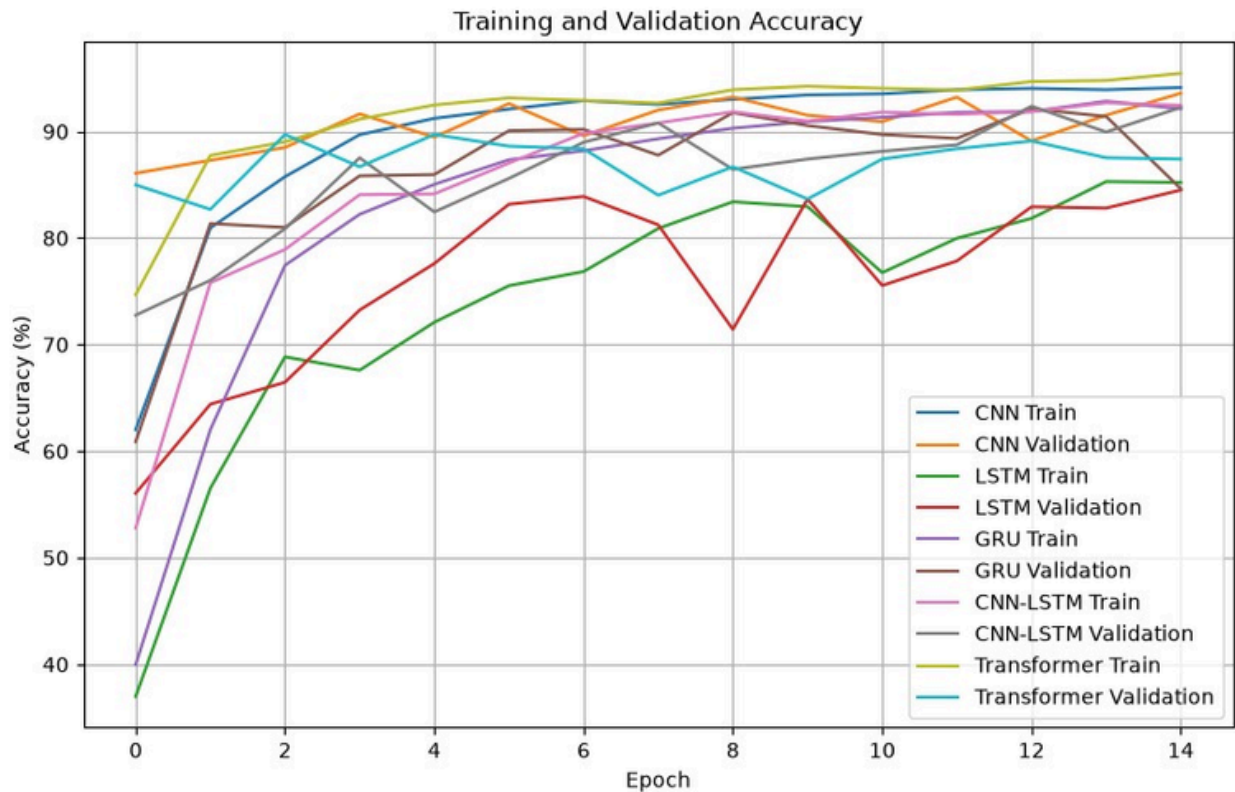


Figure 1. Training and validation accuracy across the five models.

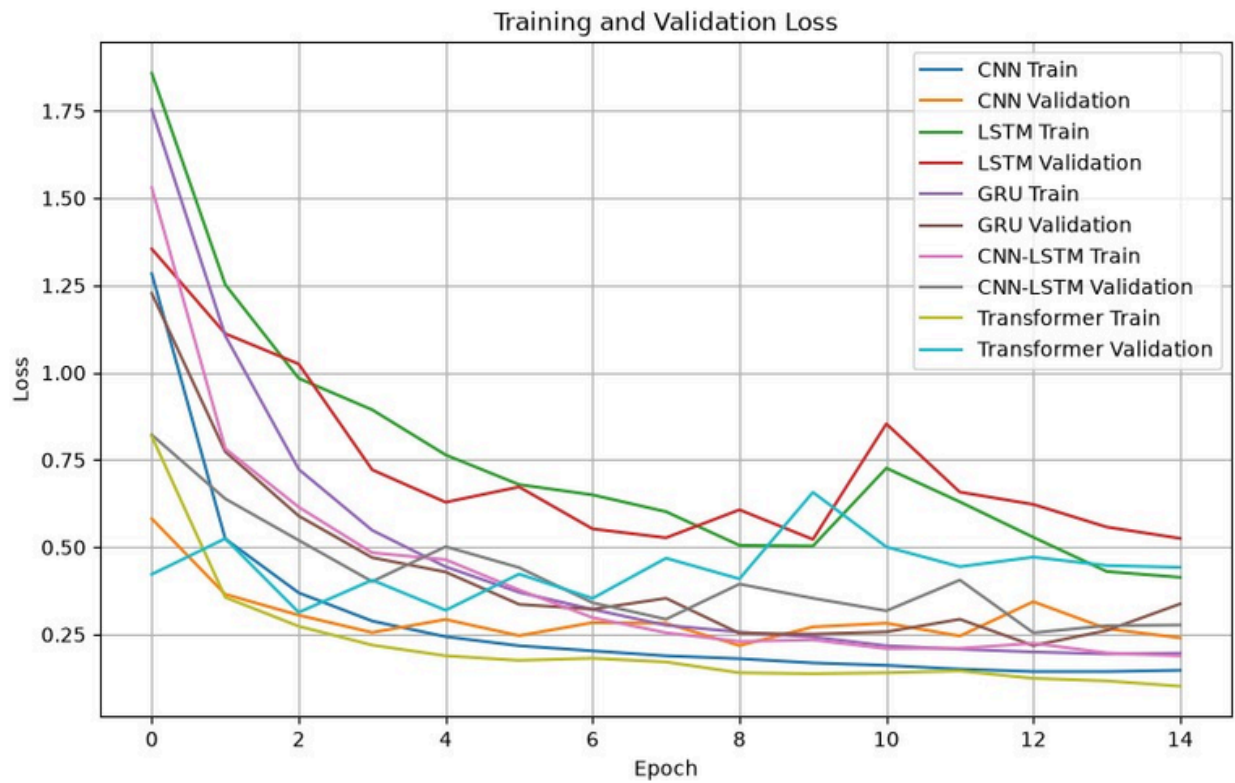


Figure 2. Training and validation loss across the five models.

The learning curves show that all models learn useful representations during training. The Transformer reaches very high training accuracy but maintains a larger gap between training and validation performance, indicating that its greater capacity does not translate directly into better test performance in this configuration.

7. Test Results and Model Comparison

7.1 Overall Performance

Model	Parameters	Train (s)	Accuracy	Precision	Recall	Weighted F1	Macro F1	Inference (s)
CNN	44,620	32.45	93.73%	93.99%	93.73%	93.65%	86.76%	0.2301
LSTM	52,492	41.06	88.43%	87.83%	88.43%	87.65%	57.60%	0.2092
GRU	39,564	149.74	86.69%	87.94%	86.69%	86.40%	73.92%	0.5538
CNN-LSTM	89,868	29.45	92.96%	93.32%	92.96%	92.83%	79.60%	0.1790
Transformer	563,532	2295.26	91.13%	90.37%	91.13%	90.52%	69.73%	2.3592

Table 6. Final model comparison from the project notebook.

The CNN is the strongest model overall: it achieves the highest test accuracy (93.73%) and Macro F1 (86.76%). CNN-LSTM is a close second in accuracy at 92.96% and has the shortest measured inference time at 0.1790 s. The Transformer has the largest parameter count and by far the longest training and inference times, while its 91.13% accuracy remains below both CNN and CNN-LSTM.

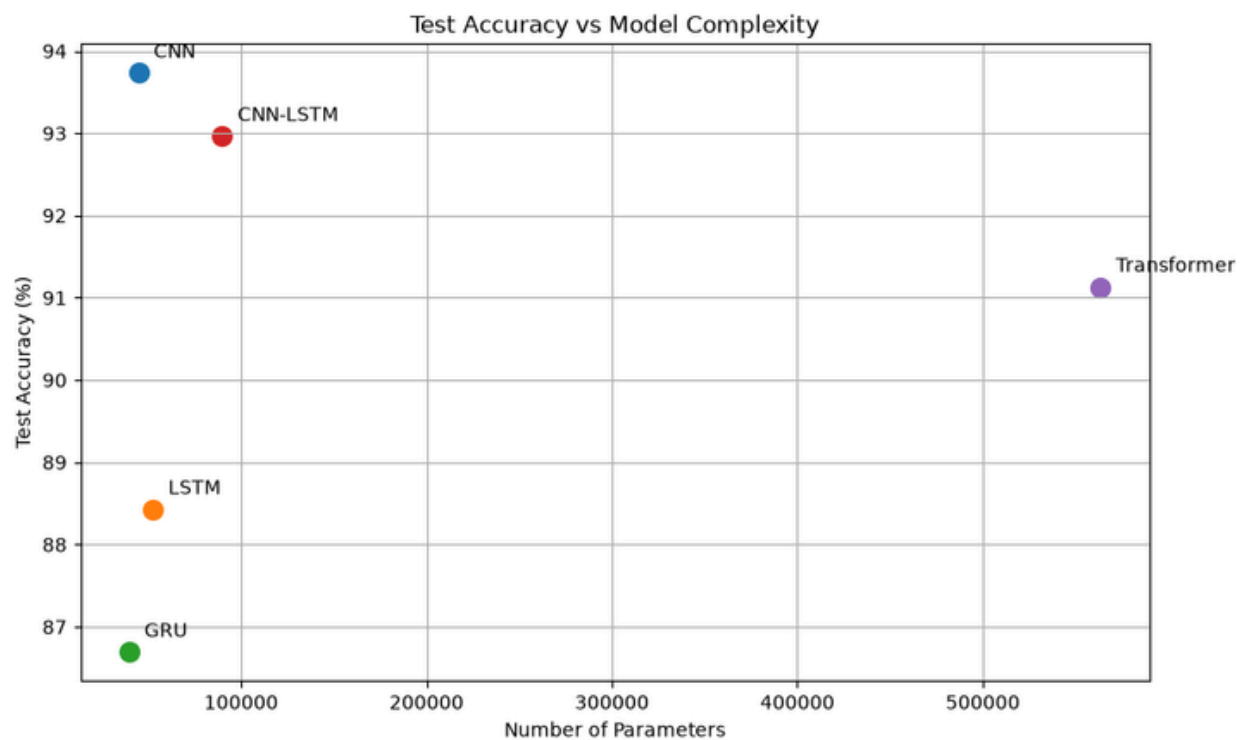


Figure 5. Test accuracy versus model complexity.

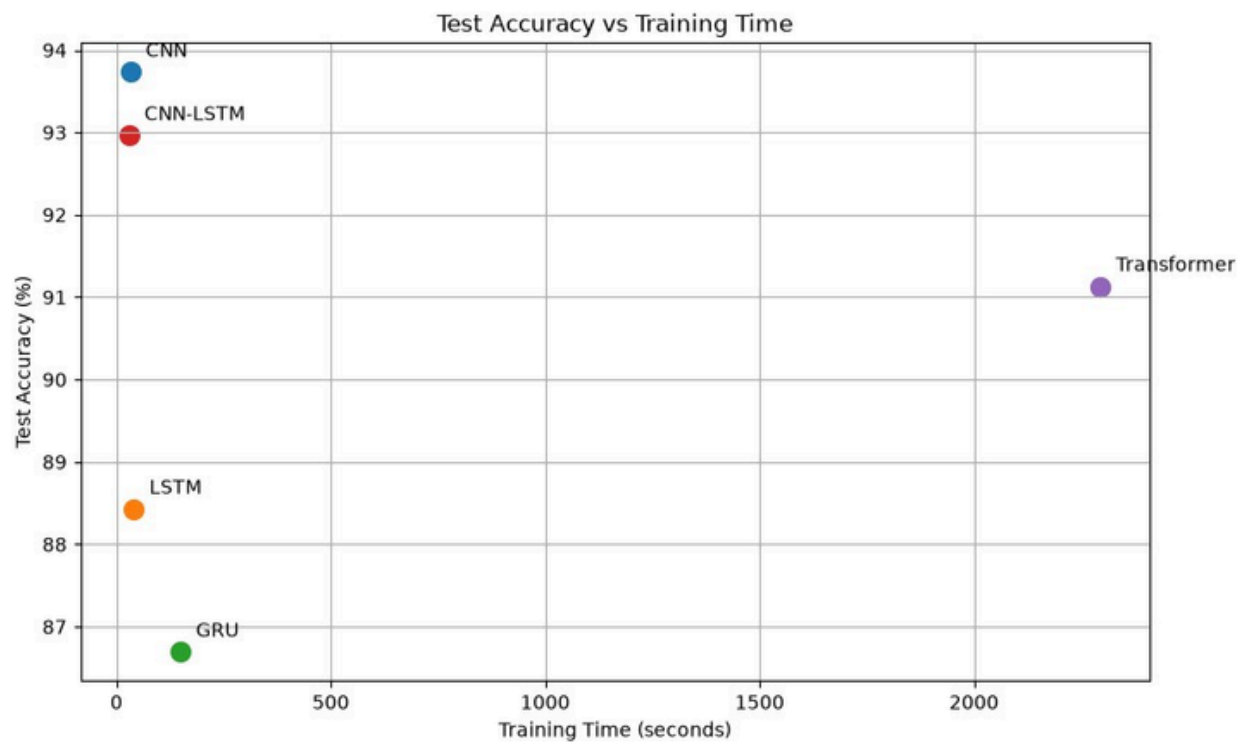


Figure 6. Test accuracy versus training time.

8. CNN Classification Analysis

8.1 Confusion Matrix

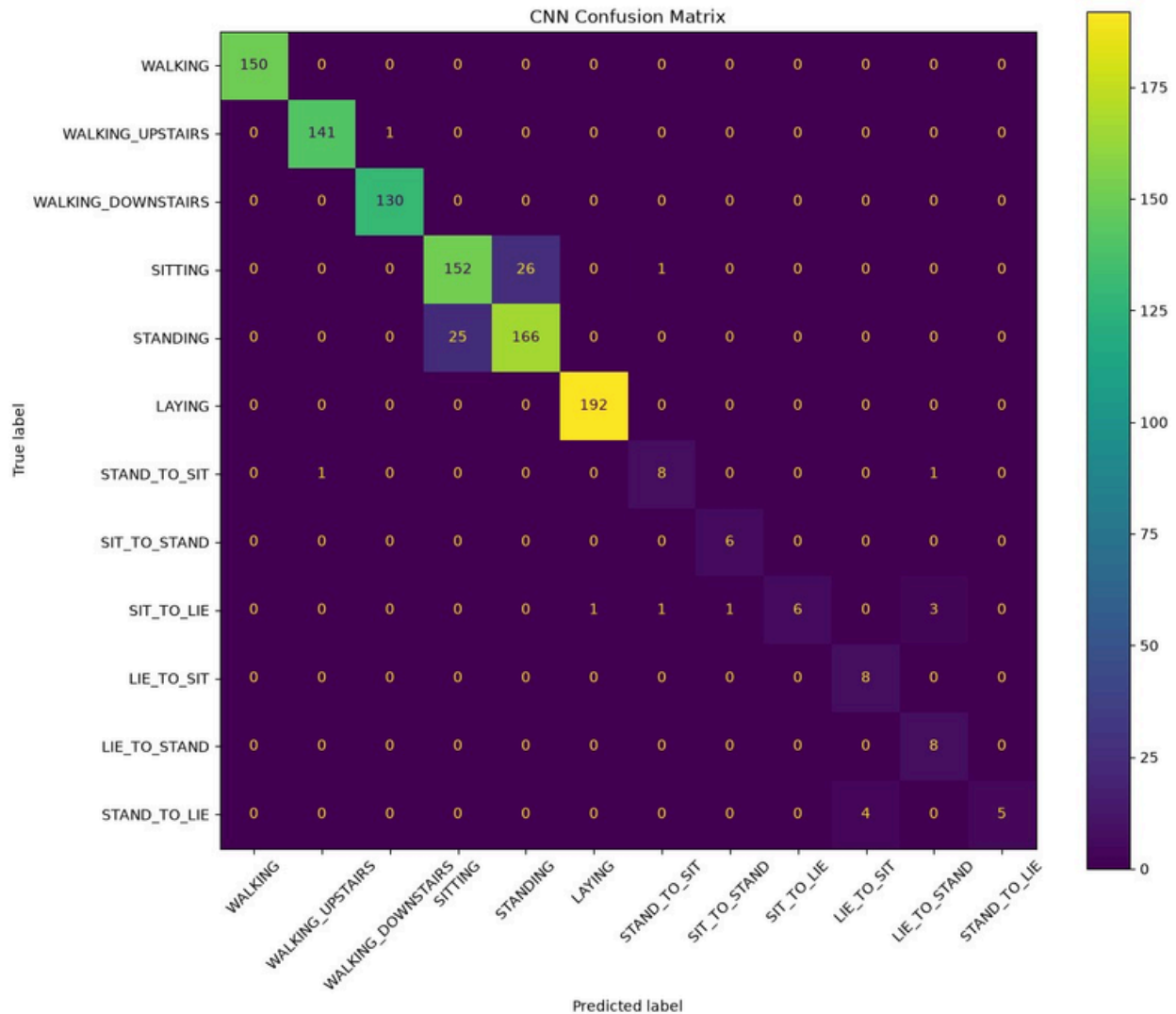


Figure 3. CNN confusion matrix on the 1,037-sample test set.

The CNN confusion matrix shows very strong recognition of walking-related classes and laying. The largest visible confusions among the common classes occur between sitting and standing, which is consistent with the similarity of their sensor patterns.

8.2 CNN Classification Report

Activity	Precision	Recall	F1	Support
WALKING	1.0000	1.0000	1.0000	150
WALKING_UPSTAIRS	0.9930	0.9930	0.9930	142
WALKING_DOWNSTAIRS	0.9924	1.0000	0.9962	130
SITTING	0.8588	0.8492	0.8539	179
STANDING	0.8646	0.8691	0.8668	191
LAYING	0.9948	1.0000	0.9974	192
STAND_TO_SIT	0.8000	0.8000	0.8000	10

SIT_TO_STAND	0.8571	1.0000	0.9231	6
SIT_TO_LIE	1.0000	0.5000	0.6667	12
LIE_TO_SIT	0.6667	1.0000	0.8000	8
LIE_TO_STAND	0.6667	1.0000	0.8000	8
STAND_TO_LIE	1.0000	0.5556	0.7143	9

Table 7. CNN per-class classification report.

The CNN achieved an overall accuracy of 93.73%, weighted F1 of 93.65%, and Macro F1 of 86.76%. The gap between weighted and Macro F1 reflects the class imbalance: the CNN is highly accurate on the common classes, while some rare transition classes have lower F1-scores.

9. Per-Class Comparison Across Models

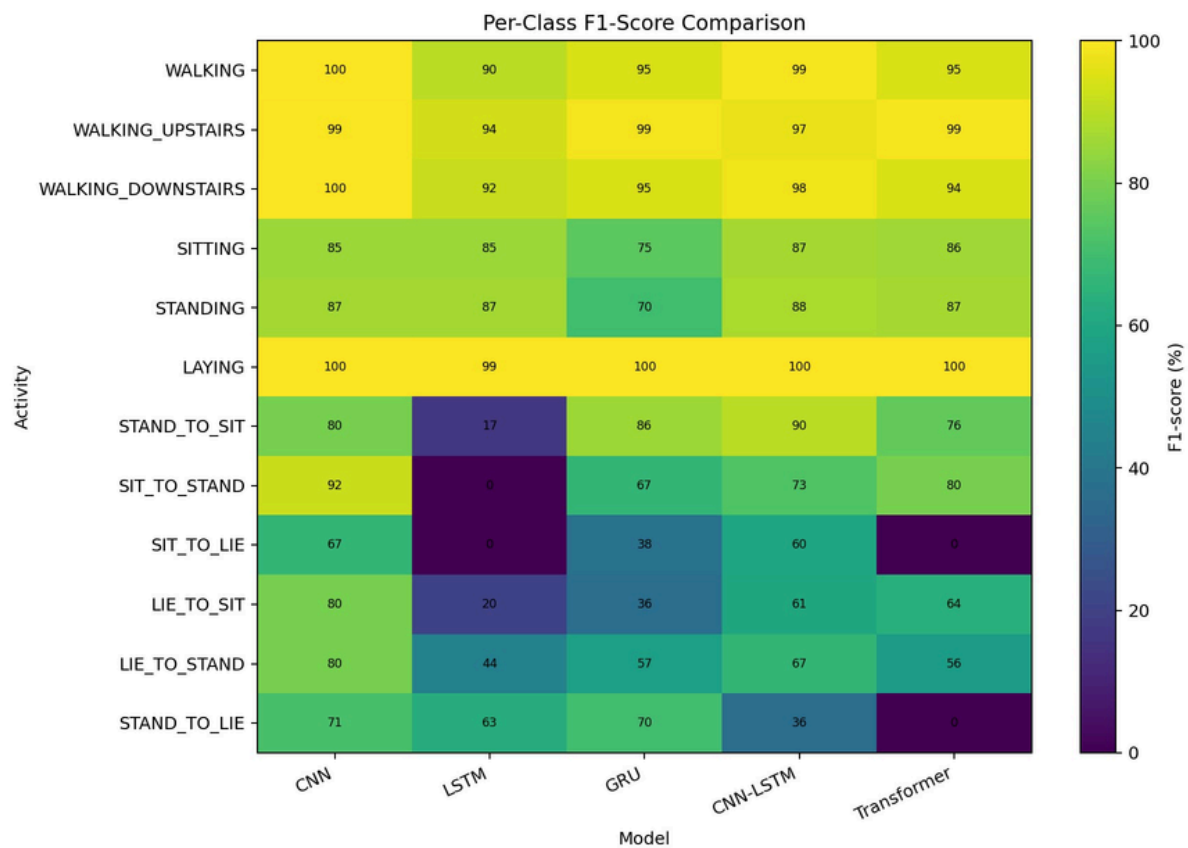


Figure 4. Per-class F1-score comparison across all five models, generated from the notebook's classification reports.

Activity	CNN	LSTM	GRU	CNN-LSTM	Transformer
WALKING	100.00%	90.22%	94.90%	99.00%	94.57%
WALKING_UPSTAIRS	99.30%	93.66%	98.59%	97.22%	98.95%
WALKING_DOWNSTAIRS	99.62%	91.87%	94.74%	97.67%	94.31%
SITTING	85.39%	85.25%	75.30%	87.05%	86.10%
STANDING	86.68%	86.70%	69.75%	87.83%	86.93%
LAYING	99.74%	99.22%	99.74%	99.74%	99.74%

STAND_TO_SIT	80.00%	16.67%	85.71%	90.00%	76.19%
SIT_TO_STAND	92.31%	0.00%	66.67%	72.73%	80.00%
SIT_TO_LIE	66.67%	0.00%	38.10%	60.00%	0.00%
LIE_TO_SIT	80.00%	20.00%	36.36%	60.87%	64.00%
LIE_TO_STAND	80.00%	44.44%	57.14%	66.67%	56.00%
STAND_TO_LIE	71.43%	63.16%	70.00%	36.36%	0.00%

Table 8. Per-class F1-score comparison.

The CNN is the best model for most activities. CNN-LSTM performs best on sitting, standing, and stand-to-sit, while the CNN is particularly strong on walking, walking upstairs, walking downstairs, laying, sit-to-stand, sit-to-lie, lie-to-sit, lie-to-stand, and stand-to-lie. Rare transition classes should be interpreted cautiously because their test supports are small.

10. Sensor Ablation Study

To determine the contribution of each sensor modality, the project trains CNN variants using accelerometer-only input, gyroscope-only input, and the combined six-channel input. The accelerometer and gyroscope models use the same CNN architecture with three input channels instead of six.

Configuration	Parameters	Train (s)	Accuracy	Precision	Recall	Weighted F1	Macro F1	Inference (s)
Accelerometer Only	43,660	32.64	91.90%	91.73%	91.90%	91.23%	69.14%	0.3307
Gyroscope Only	43,660	32.91	72.23%	74.37%	72.23%	69.77%	66.89%	0.1582
Both Sensors	44,620	32.45	93.73%	93.99%	93.73%	93.65%	86.76%	0.2301

Table 9. Final sensor-ablation results.

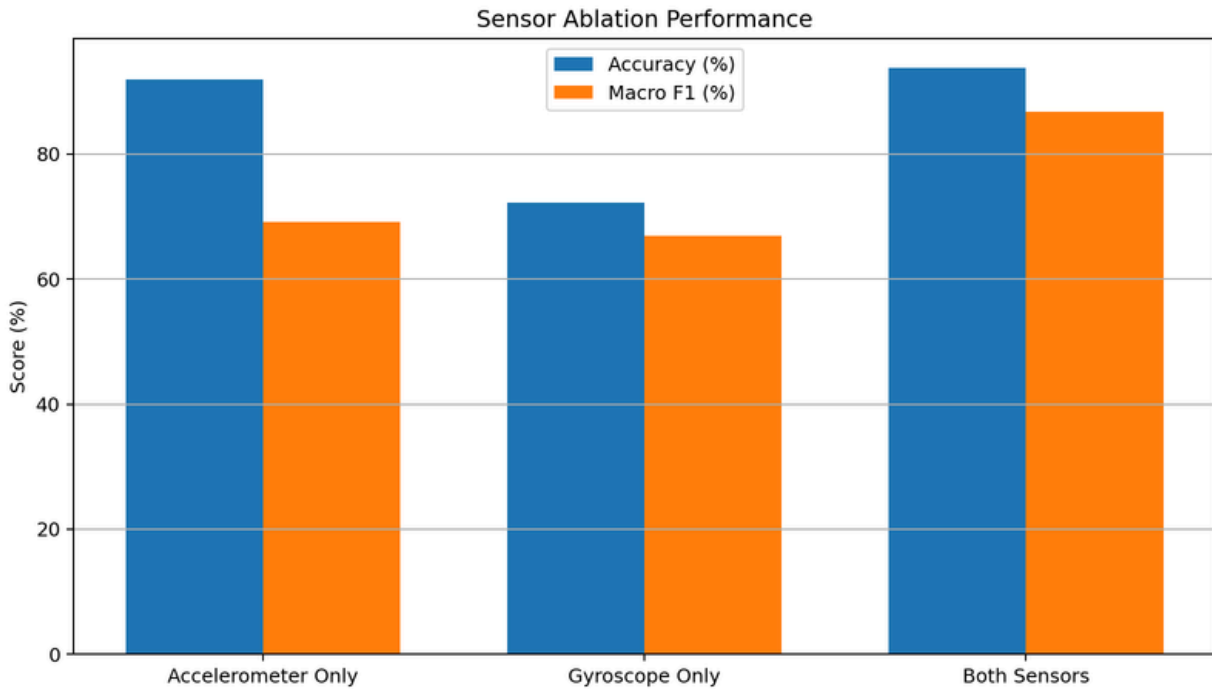


Figure 7. Sensor ablation comparison using test accuracy and Macro F1.

Accelerometer-only input reaches 91.90% accuracy and 69.14% Macro F1. Gyroscope-only input drops to 72.23% accuracy and 66.89% Macro F1. Combining both sensors increases performance to 93.73% accuracy and 86.76% Macro F1. Compared with accelerometer-only input, sensor fusion improves accuracy by 1.83 percentage points and Macro F1 by 17.62 percentage points. This demonstrates that the two modalities provide complementary information.

11. Efficiency and Ranking

11.1 Final Rankings

Model	Accuracy Rank	Macro F1 Rank	Parameter Rank	Training Rank	Inference Rank
CNN	1	1	2	2	3
CNN-LSTM	2	2	4	1	1
Transformer	3	4	5	5	5
LSTM	4	5	3	3	2
GRU	5	3	1	4	4

Table 10. Final model rankings reported by the project notebook.

The ranking shows a clear trade-off. CNN has the best recognition performance, CNN-LSTM has the fastest training and inference, and GRU has the fewest parameters. However, the smallest model is not necessarily the most efficient in runtime: the GRU has fewer parameters than CNN but takes substantially longer to train and infer in this experiment.

12. Discussion

12.1 Best Overall Model

CNN is the recommended model under the tested configuration. It combines the highest accuracy and Macro F1 with a moderate parameter count and short training time. Its convolutional structure is well suited to extracting local temporal patterns from inertial signals.

12.2 CNN-LSTM Trade-off

CNN-LSTM achieves 92.96% accuracy, only 0.77 percentage points below CNN, while recording the shortest training and inference times among the five main models. Its Macro F1 of 79.60%, however, is lower than CNN's 86.76%, so CNN remains preferable when class-balanced performance is important.

12.3 Recurrent Models

LSTM reaches 88.43% accuracy but has a low Macro F1 of 57.60%, indicating weaker performance on minority classes. GRU has the smallest model size at 39,564 parameters and a Macro F1 of 73.92%, but its measured training and inference times are relatively high.

12.4 Transformer

The Transformer reaches 91.13% accuracy and 69.73% Macro F1. Its 563,532 parameters and 2,295.26-second training time make it substantially more expensive than the other tested models. In this experiment, the additional model capacity does not produce a performance advantage.

12.5 Class Imbalance

The difference between weighted F1 and Macro F1 is an important result. CNN weighted F1 is 93.65%, whereas Macro F1 is 86.76%. This means the model is highly effective on common classes but less consistent across the rare transition classes. Future work should therefore consider imbalance-aware training and additional samples for transitions.

13. Limitations and Future Work

- The processed windows are strongly class-imbalanced, especially for postural transitions.
- Several transition classes have very small test supports, so individual F1-scores are sensitive to a small number of predictions.
- The comparison reflects the exact architectures, hyperparameters, 15-epoch schedule, and CPU execution used in the project; it does not represent the theoretical maximum performance of each architecture.
- The experiment uses a fixed subject split (users 1–21, 22–25, and 26–30). Additional cross-validation or alternative subject splits could provide a broader robustness assessment.
- Future work could investigate class-weighted or focal losses, oversampling, augmentation of minority transitions, and systematic hyperparameter tuning.

- For deployment, model quantization, pruning, and mobile inference benchmarking could be explored.

14. Conclusion

This project implemented and compared CNN, LSTM, GRU, CNN-LSTM, and Transformer models for smartphone-based human activity and postural-transition recognition using raw accelerometer and gyroscope signals. The data were converted into 128-step windows with six sensor features and evaluated using a subject-based train/validation/test split.

The CNN using both sensor modalities produced the best overall result: 93.73% test accuracy, 93.99% precision, 93.73% recall, 93.65% weighted F1, and 86.76% Macro F1 with 44,620 trainable parameters. The sensor-ablation experiment confirmed that accelerometer information is substantially stronger than gyroscope information when used alone, while the combined representation provides the strongest overall and class-balanced performance.

Under the exact experimental conditions of this project, CNN is therefore the recommended final model. CNN-LSTM is a strong alternative when the shortest measured training and inference time is prioritized, while the Transformer does not justify its substantially higher computational cost in this configuration.

15. References

[1] J. Reyes-Ortiz, D. Anguita, L. Oneto, and X. Parra, "Smartphone-Based Recognition of Human Activities and Postural Transitions," UCI Machine Learning Repository, 2015. doi: 10.24432/C54G7M.

Official dataset record: UCI Machine Learning Repository, dataset 341. The repository documents the raw triaxial accelerometer and gyroscope signals, the 12 activity/postural-transition labels, and the 50 Hz sampling setup.

Appendix A. Reproducibility Summary

The project notebook used for this report is 01_Data_Preparation.ipynb. The notebook contains the data loading, window construction, subject-based split, normalization, PyTorch DataLoaders, five model definitions, training loops, test evaluation, classification reports, confusion matrix, model-comparison plots, and sensor-ablation results.

Component	Project setting
Raw sensor channels	Accelerometer X/Y/Z + Gyroscope X/Y/Z
Sequence length	128 samples
Classes	12
Train subjects	1–21
Validation subjects	22–25
Test subjects	26–30
Batch size	64

Epochs	15
Loss	CrossEntropyLoss
Optimizer	Adam, lr=0.001
Execution device	CPU